

Google Play Store Reviews

Natural Language Processing



Main Objective of Analysis

Recently, generative AI models such as OpenAI's ChatGPT, Google's Gemini, and Microsoft's Copilot have been garnering a lot of attention. This technology works by taking in a text input and generating a text output based on the prompt. These models utilize sophisticated natural language processing (NLP) using transformers. This analysis uses more fundamental NLP approaches to predict whether the text of a Google Play Store review is in a Positive, Negative, or Neutral sentiment category. The corpus of the text was preprocessed and on top of traditional machine learning, deep learning was also implemented and the resulting metrics were compared to determine which model performed the best.

Description of Dataset

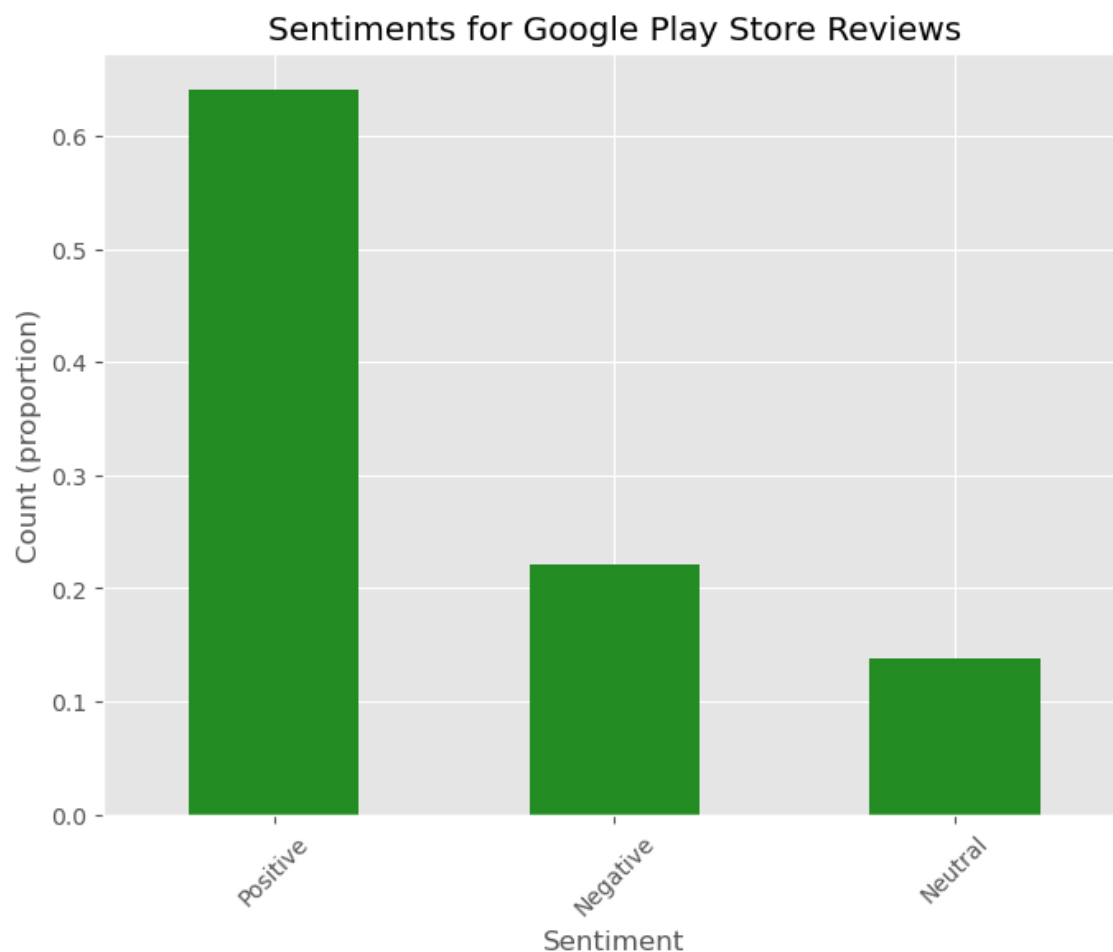
The reviews were originally in a non-English language, so the text was already translated into English in the dataset used for the analysis. 64,295 entries were in the dataset which would later be reduced in the preprocessing stage.

- App (object)
- Translated_Review (object)
- Sentiment (object)
- Sentiment_Polarity (float64)
- Sentiment_Subjectivity (float64)

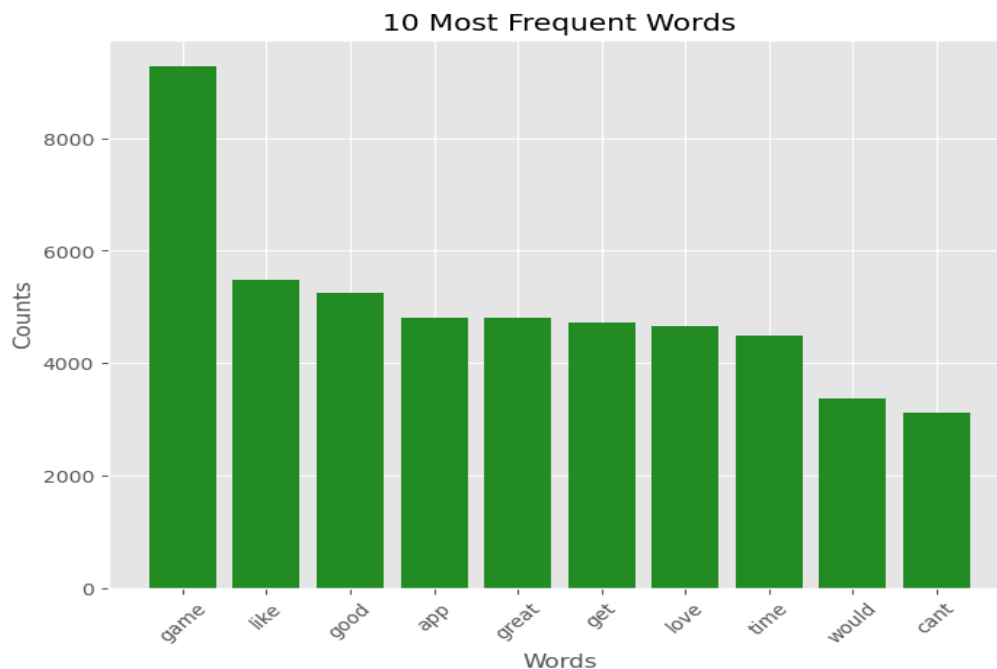
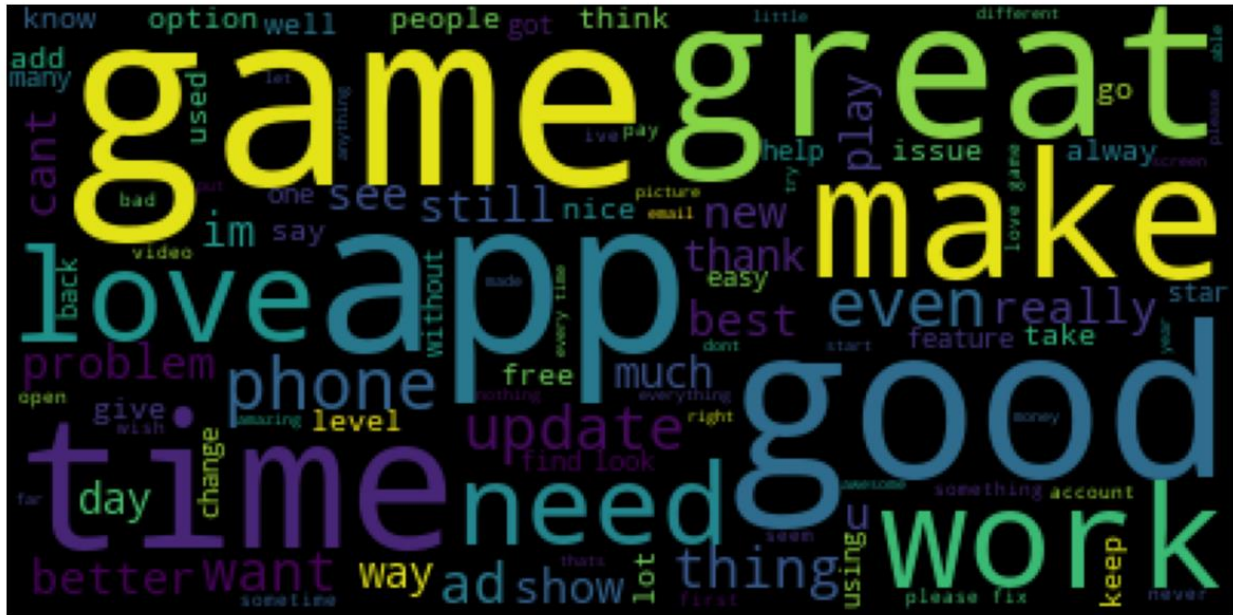
Exploratory Data Analysis, Data Cleaning, and Feature Engineering

App, Sentiment_Polarity, and Sentiment_Subjectivity were irrelevant attributes, so they were dropped column-wise. Null values were also cleaned by dropping them, as enough entries would be left over afterwards for analysis and predictive models. Regular expressions were used to: lowercase the text, replace some symbols by a space, delete bad symbols, html addresses, and stopwords. In addition, regular expressions were used to delete emojis from the text of the reviews. The individual reviews were combined into a corpus. TreebankWordTokenizer() was used to tokenize the text. To vectorize the text, CountVectorizer() and TfidfVectorizer() were utilized and in building predictive models, both would be used per model to compare which would lead to better performance metrics. LabelEncoder() was performed to encode the categories and a 80/20 split was used to separate the train and test sets.

In the dataset, the most numerous category was positive reviews, compared to negative ones. The neutral category occurred least frequently, but was still used for the analysis.



A word cloud was generated to provide a quick and easy visualization to get an idea of which key words appear the most in the corpus. The frequency of certain words is the basis for the vectorization methods used to build predictive models. To complement the word cloud, a bar chart shows the 10 most frequently occurring words in the corpus.



The following is a comparison between (a) original text and (b) preprocessed text after the data was cleaned.

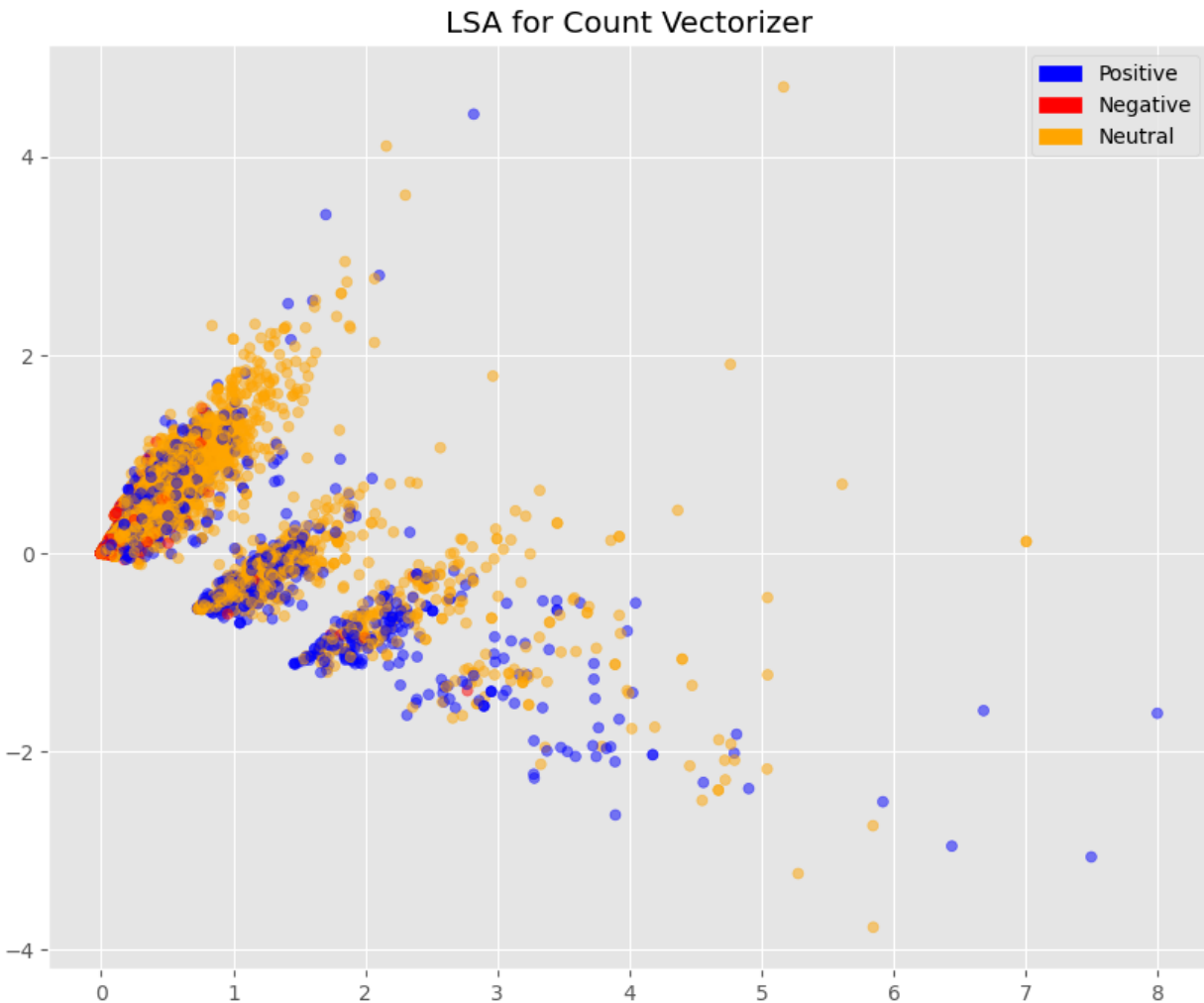
(a) Original text of reviews

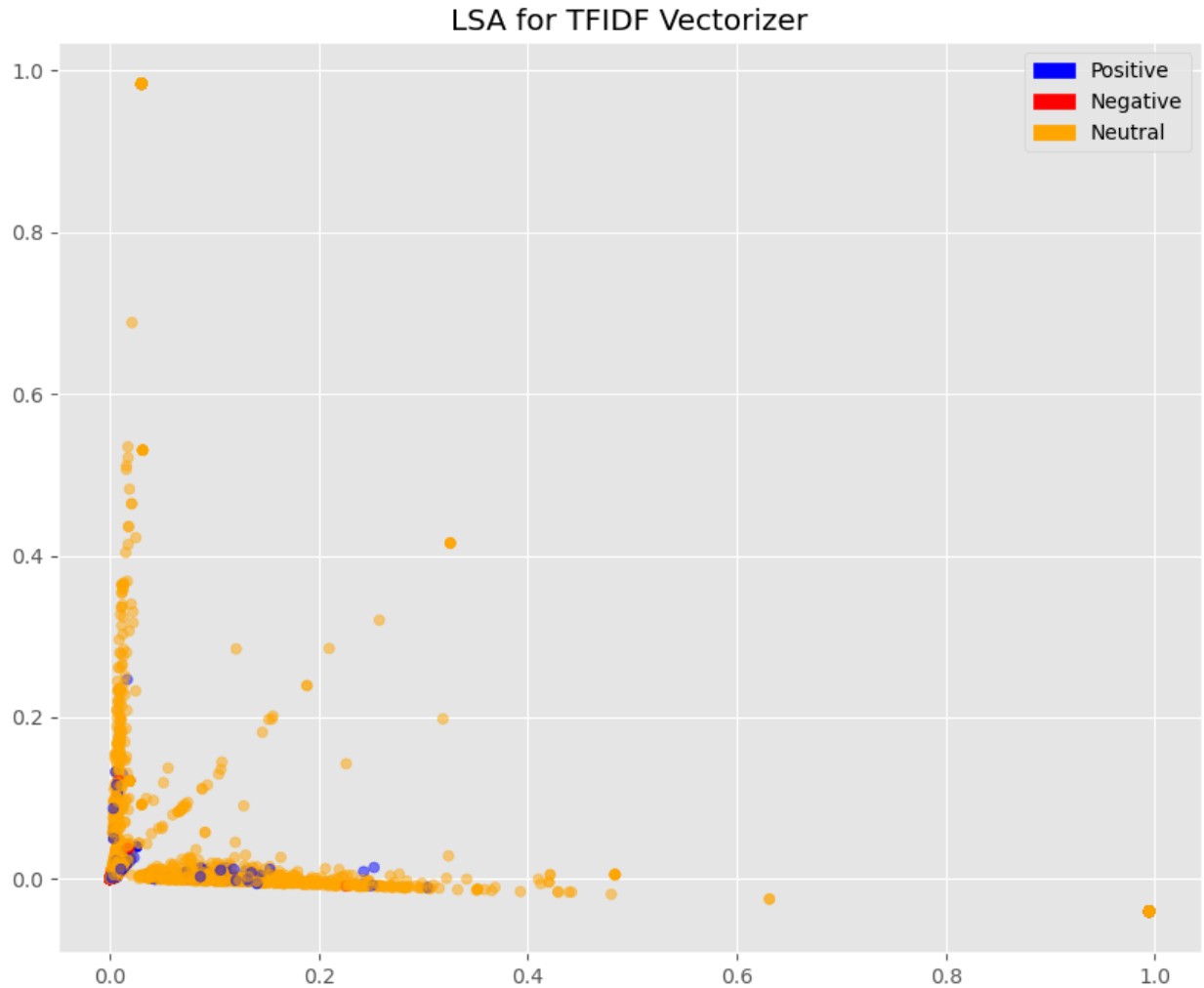
```
0 I like eat delicious food. That's I'm cooking ...
1 This help eating healthy exercise regular basis
2 Works great especially going grocery store
3 Best idea us
4 Best way
Name: Translated_Review, dtype: obj
```

(b) Preprocessed text of reviews

```
0 like eat delicious food thats im cooking food ...
1 help eating healthy exercise regular basis
2 works great especially going grocery store
3 best idea us
4 best way
Name: Translated_Review, dtype: object
```

Latent Semantic Analysis (LSA) plots were made to compare concepts based on the terms in the corpus for both count vectorizer and TFIDF vectorizer. The spread of terms in different categories for count vectorizer is wider, compared to the TFIDF vectorizer, which is compressed and the categories for the terms using count vectorizer are much more cleanly grouped together. Count vectorizer seems to capture the latent semantic structure in the data better.





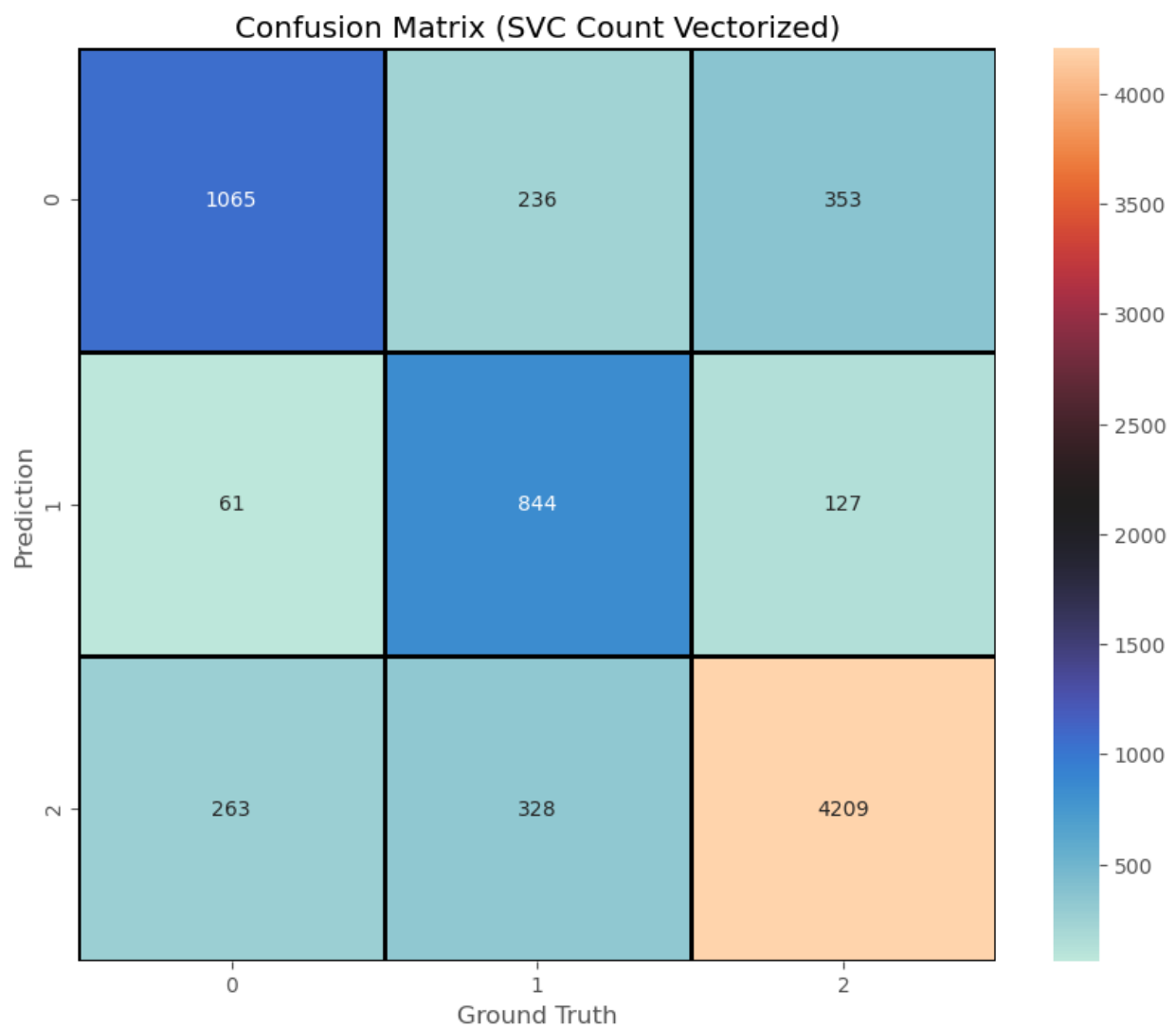
Traditional Classification Models

Three classification models were trained to predict the sentiment, for both the count vectorized and TFIDF vectorized data: support vector machine, stochastic gradient descent, and XGBoost. As indicated by the LSA plots, the models trained on the count vectorized data resulted in the optimal metrics. The two best performing algorithms, stochastic gradient descent and XGBoost were then trained on GloVe word embeddings rather than the vectors from the dataset as the number of terms in GloVe word embeddings is significantly larger. Models trained on the original dataset had superior results and one of the reasons may be because the reviews are translated into English from a non-English language and GloVe word embeddings are specifically for English, so the vectors from the text of the original dataset was better able to capture patterns in a corpus that was created from grammatically incorrect reviews. Out of these NLP models, stochastic gradient descent from count vectorized data from the corpus produced the best results.

Support Vector Machine

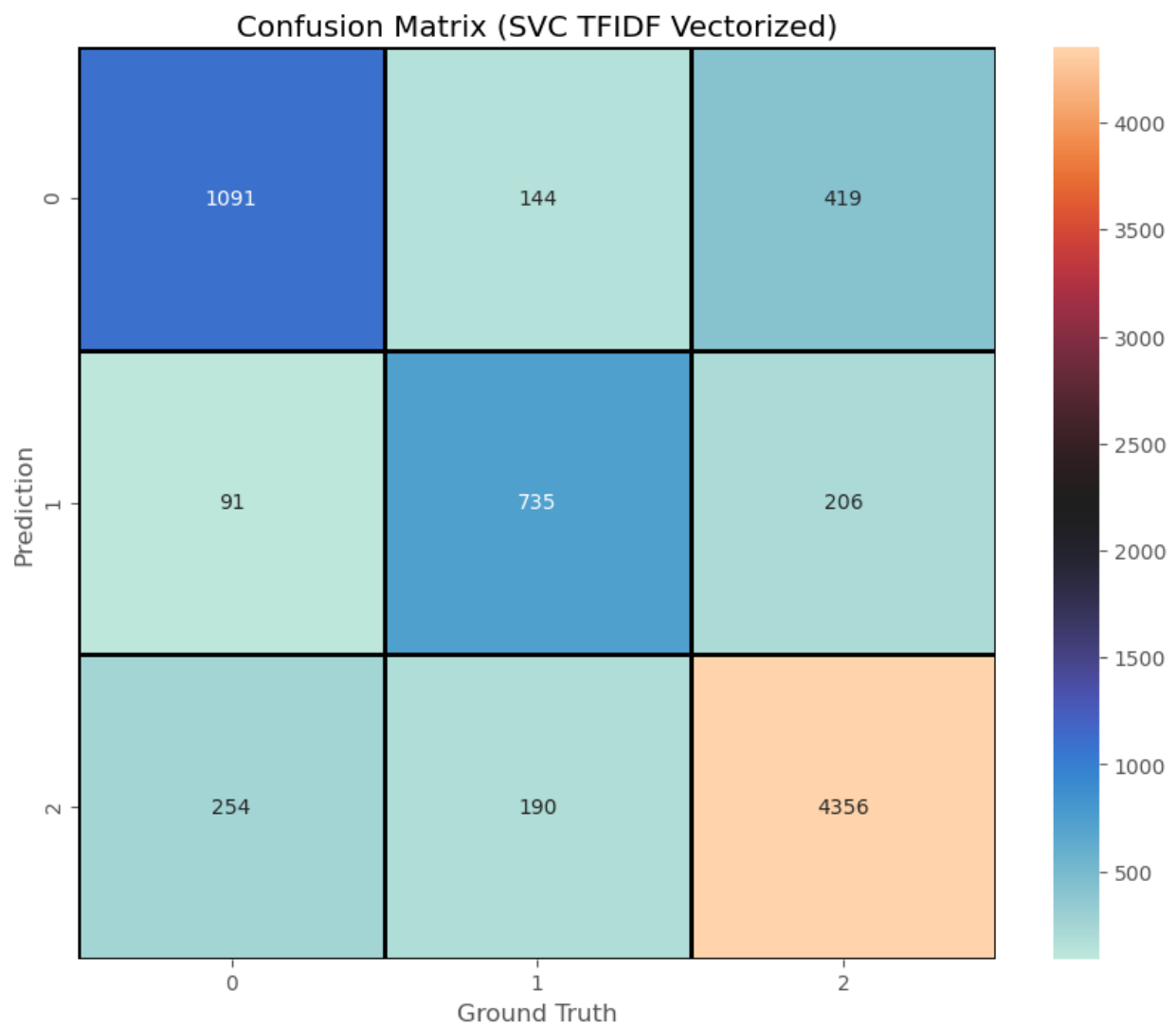
Count Vectorization

	precision	recall	f1-score	support
0	0.77	0.64	0.70	1654
1	0.60	0.82	0.69	1032
2	0.90	0.88	0.89	4800
accuracy			0.82	7486
macro avg	0.75	0.78	0.76	7486
weighted avg	0.83	0.82	0.82	7486



TFIDF Vectorization

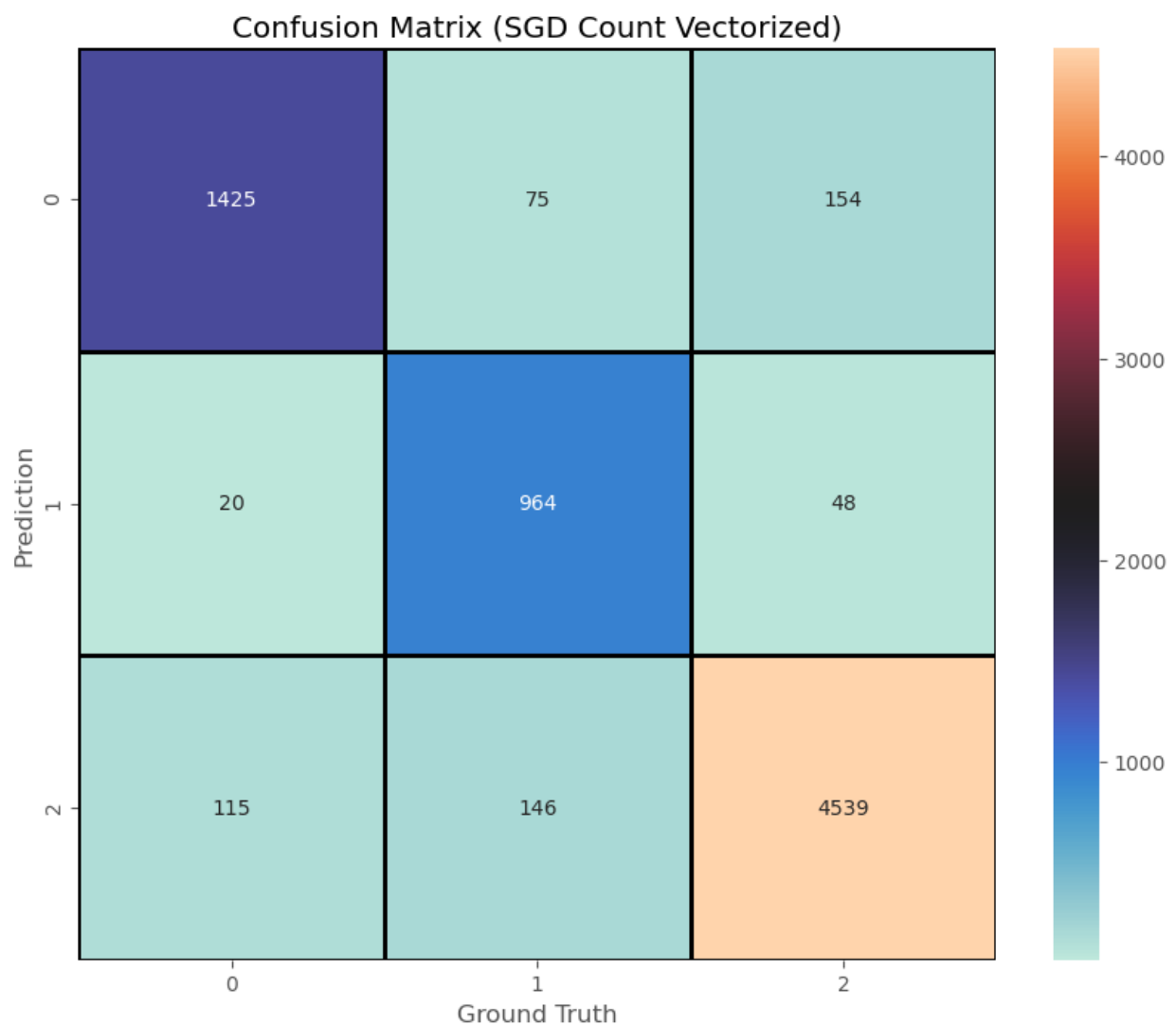
	precision	recall	f1-score	support
0	0.76	0.66	0.71	1654
1	0.69	0.71	0.70	1032
2	0.87	0.91	0.89	4800
accuracy				0.83 7486
macro avg				0.77 0.76 0.77 7486
weighted avg				0.82 0.83 0.82 7486



Stochastic Gradient Descent

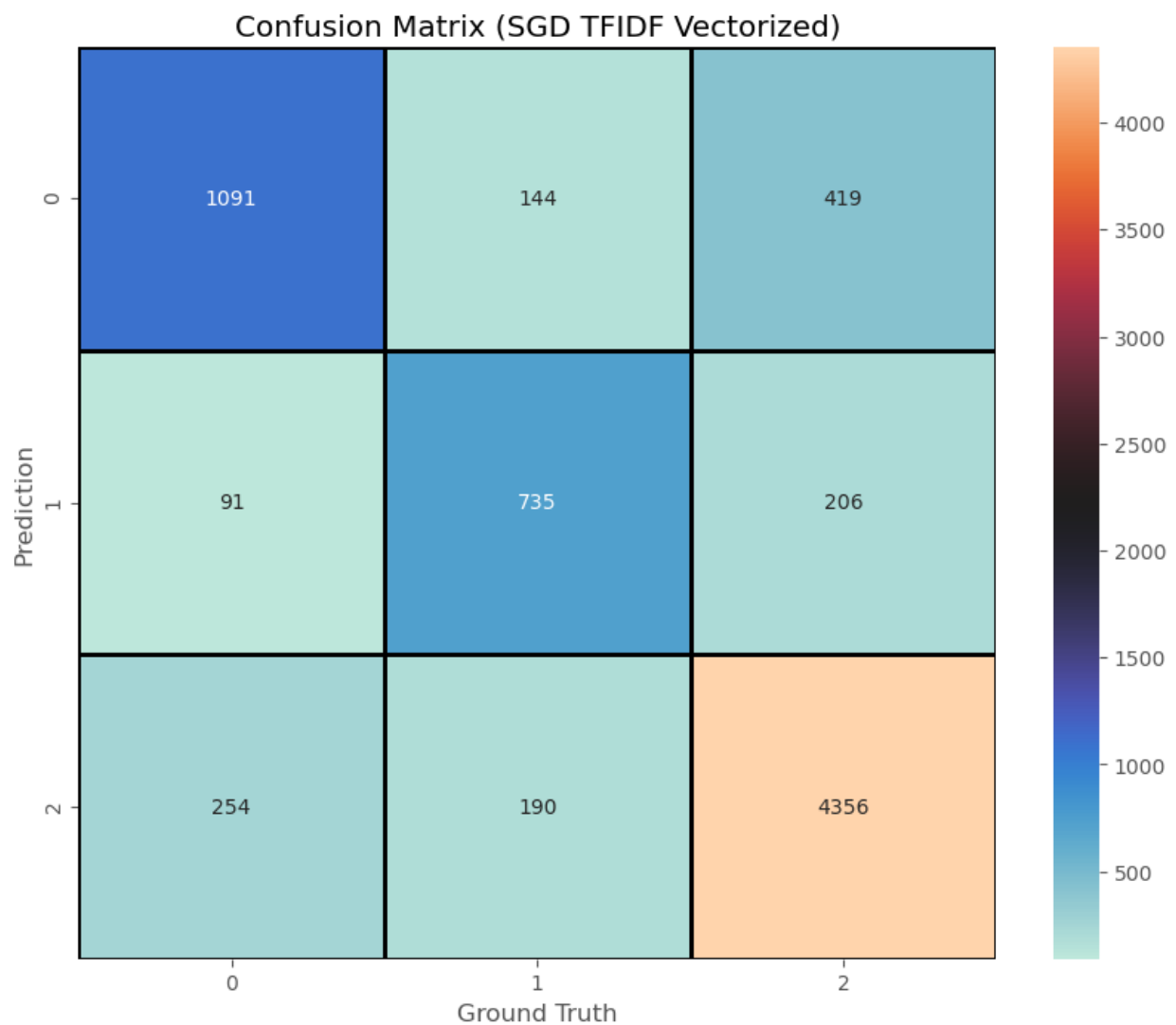
Count Vectorization

	precision	recall	f1-score	support
0	0.91	0.86	0.89	1654
1	0.81	0.93	0.87	1032
2	0.96	0.95	0.95	4800
accuracy			0.93	7486
macro avg	0.89	0.91	0.90	7486
weighted avg	0.93	0.93	0.93	7486



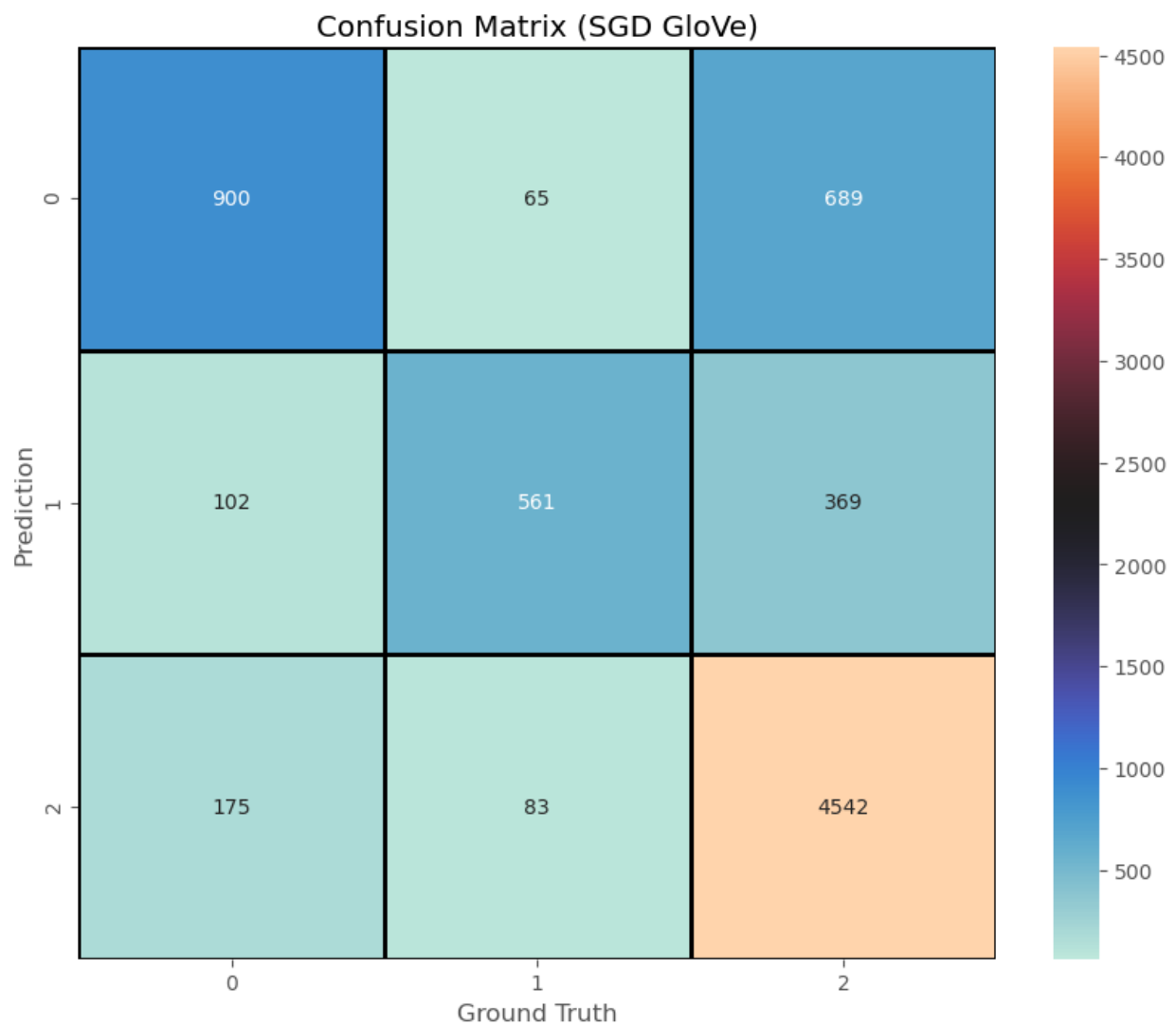
TFDIF Vectorization

	precision	recall	f1-score	support
0	0.93	0.76	0.83	1654
1	0.90	0.55	0.68	1032
2	0.85	0.98	0.91	4800
accuracy			0.87	7486
macro avg	0.89	0.76	0.81	7486
weighted avg	0.88	0.87	0.86	7486



GloVe Embeddings

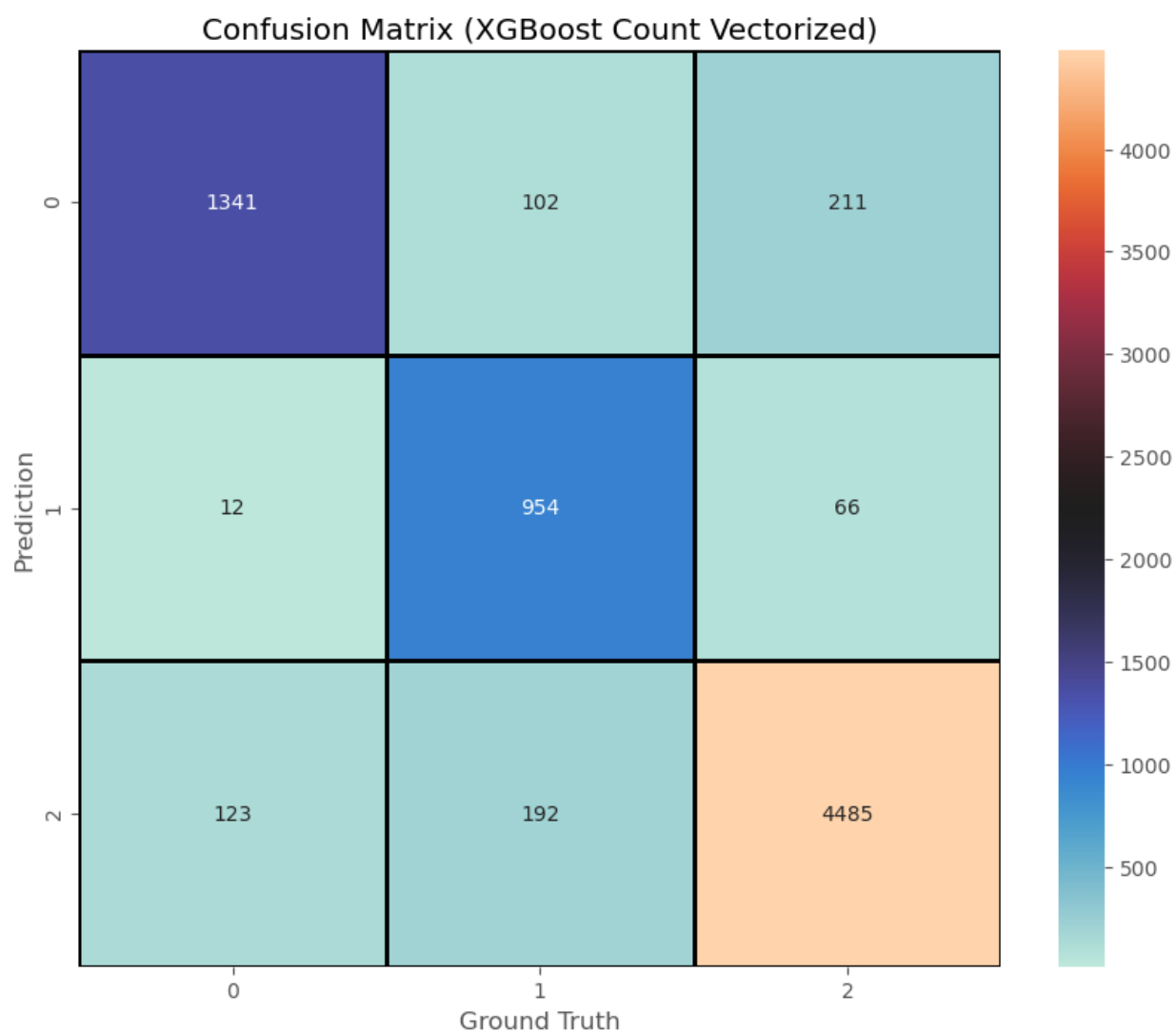
	precision	recall	f1-score	support
0	0.76	0.54	0.64	1654
1	0.79	0.54	0.64	1032
2	0.81	0.95	0.87	4800
accuracy			0.80	7486
macro avg	0.79	0.68	0.72	7486
weighted avg	0.80	0.80	0.79	7486



XGBoost

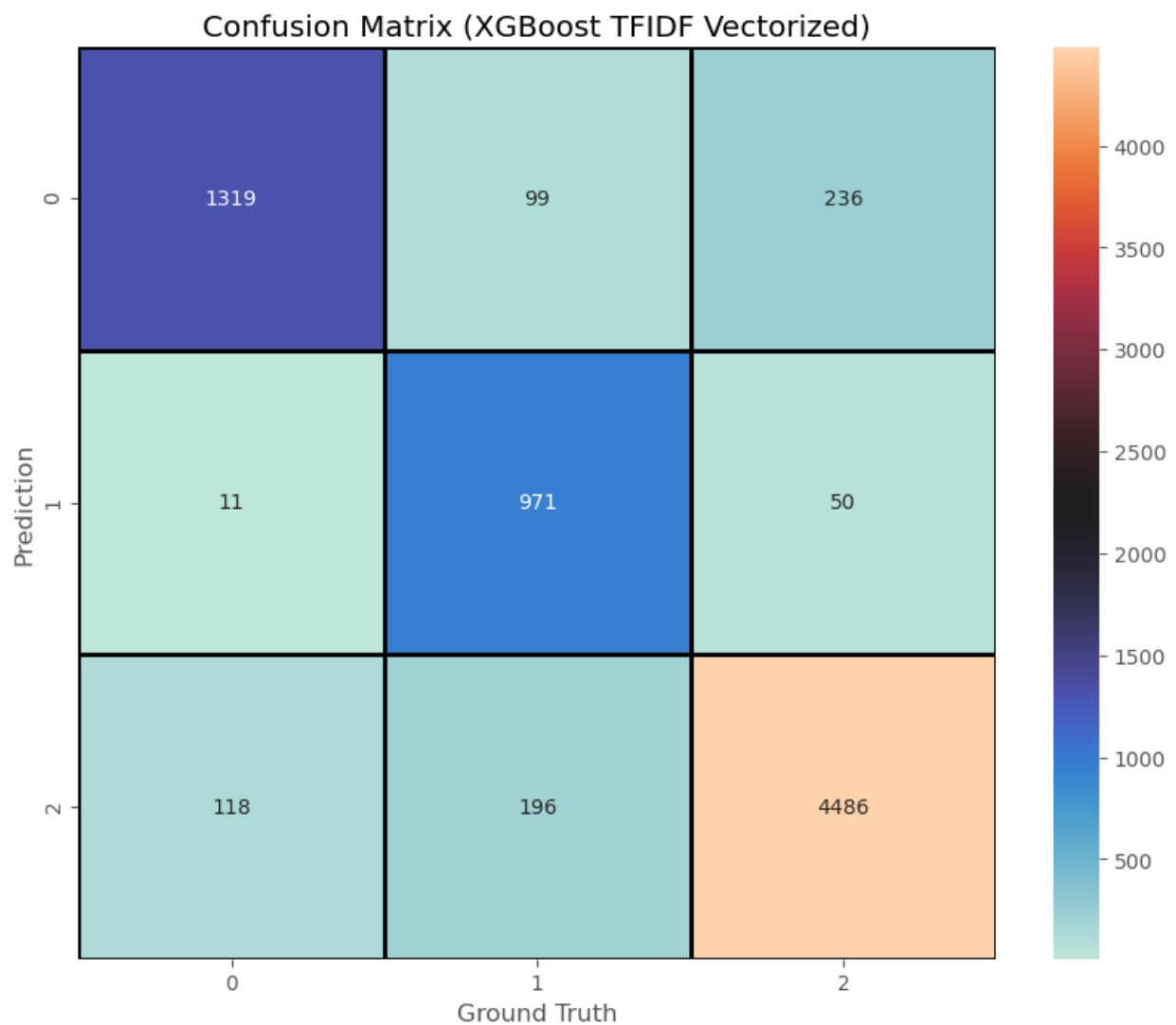
Count Vectorization

	precision	recall	f1-score	support
0	0.91	0.81	0.86	1654
1	0.76	0.92	0.84	1032
2	0.94	0.93	0.94	4800
accuracy			0.91	7486
macro avg	0.87	0.89	0.88	7486
weighted avg	0.91	0.91	0.91	7486



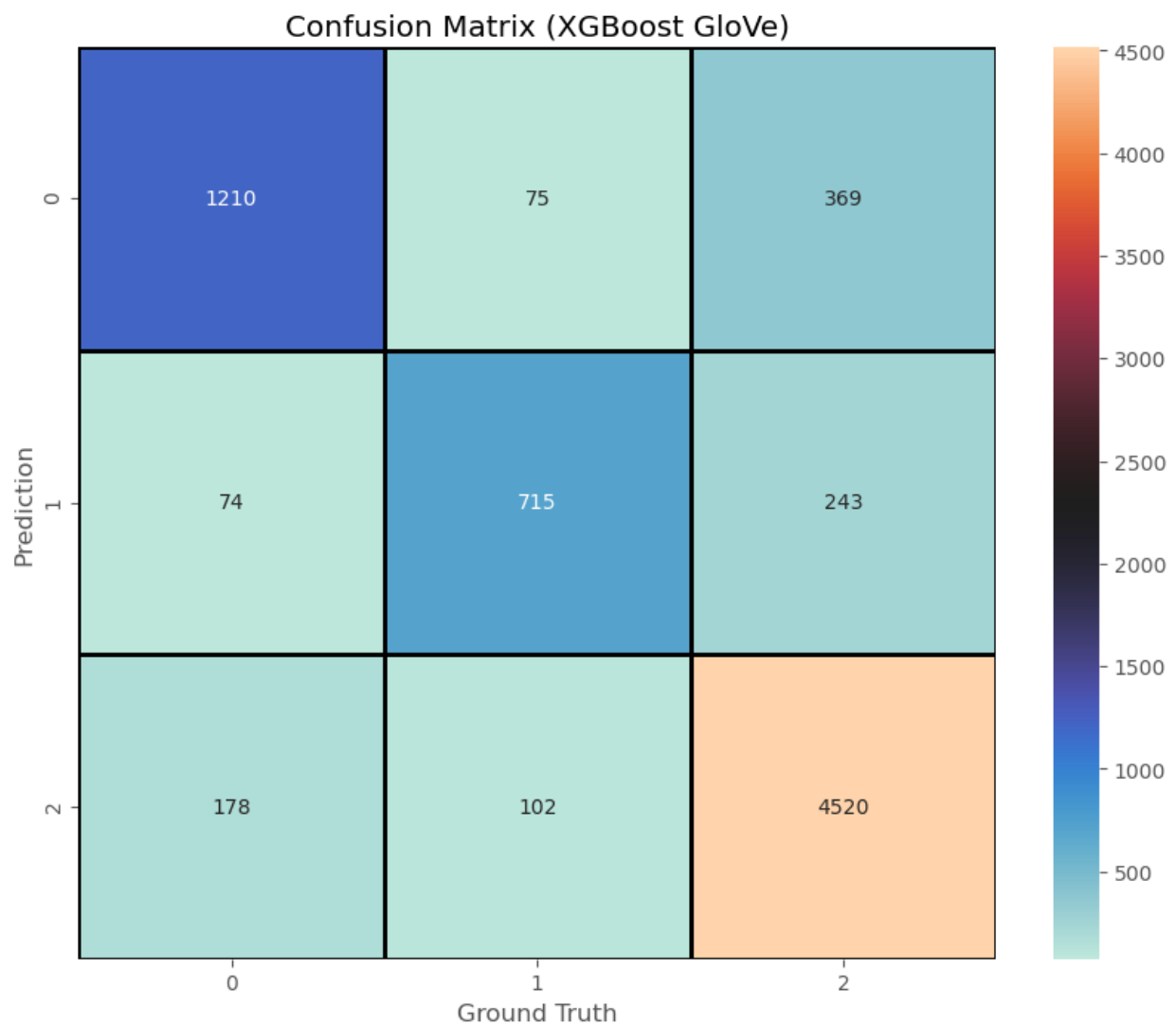
TFIDF Vectorization

	precision	recall	f1-score	support
0	0.91	0.80	0.85	1654
1	0.77	0.94	0.85	1032
2	0.94	0.93	0.94	4800
accuracy			0.91	7486
macro avg	0.87	0.89	0.88	7486
weighted avg	0.91	0.91	0.91	7486



GloVe Embeddings

	precision	recall	f1-score	support
0	0.83	0.73	0.78	1654
1	0.80	0.69	0.74	1032
2	0.88	0.94	0.91	4800
accuracy			0.86	7486
macro avg	0.84	0.79	0.81	7486
weighted avg	0.86	0.86	0.86	7486



LSTM Deep Learning Models

Long Short Term Memory (LSTM) deep learning models use a sequential neural network that allows information to persist, so they are widely used on text data for predictive models. A model was created and trained both on the embeddings from the original text as well as GloVe word embeddings. The train loss was higher than the validation loss in both cases, suggesting the model does not overfit and would generalize better to new data. Unlike the traditional machine learning models, the LSTM model trained on the GloVe embeddings performed better than the one trained on embeddings from the original corpus. However, the models from more traditional machine learning techniques such as stochastic gradient descent and XGBoost produced better scores.

Original Corpus Embeddings

Model architecture

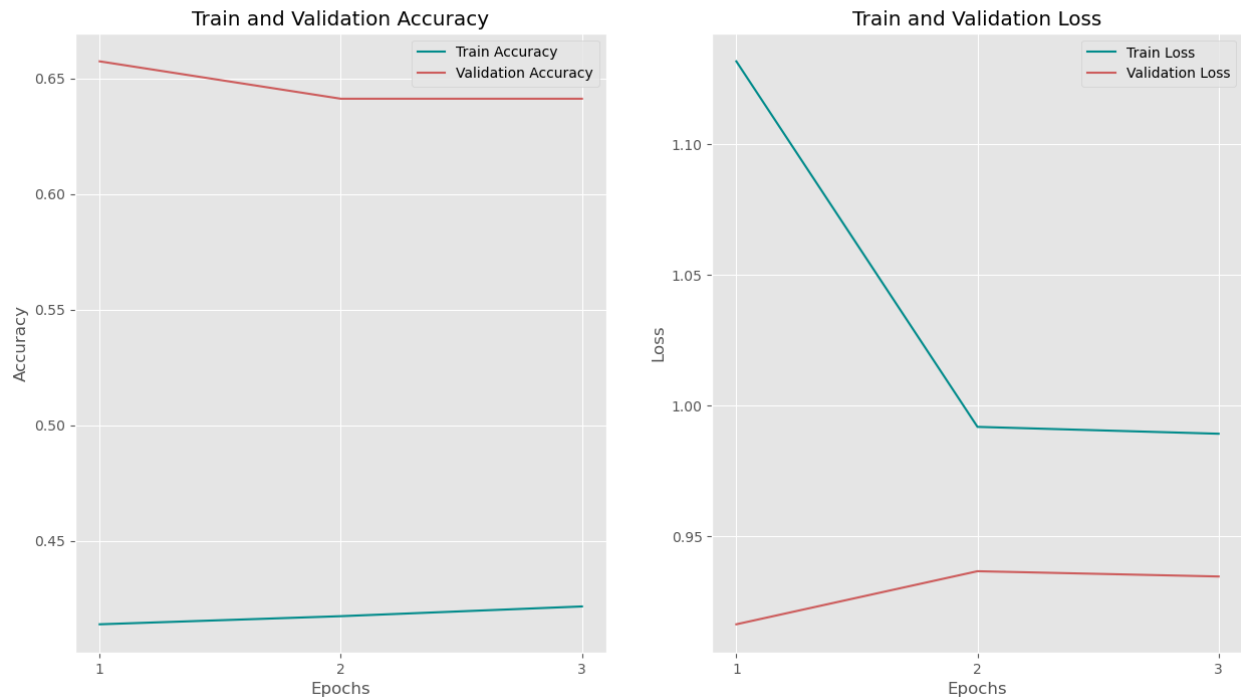
```
Model: "sequential"
Layer (type)                Output Shape              Param #
=====
embedding (Embedding)       (None, 500, 300)         7054800
spatial_dropout1d (SpatialDr (None, 500, 300)         0
bidirectional (Bidirectional (None, 600)         1442400
dense (Dense)                (None, 1024)              615424
dropout (Dropout)            (None, 1024)              0
dense_1 (Dense)              (None, 3)                 3075
dropout_1 (Dropout)          (None, 3)                 0
activation (Activation)      (None, 3)                 0
=====
Total params: 9,115,699
Trainable params: 9,115,699
Non-trainable params: 0
```

Accuracy and loss scores

```
Epoch 1/5
468/468 [=====] - 5931s 13s/step - loss: 1.1315 - accuracy: 0.4139 - val_loss: 0.9164 - val_accuracy: 0.6574
Epoch 2/5
468/468 [=====] - 5552s 12s/step - loss: 0.9919 - accuracy: 0.4174 - val_loss: 0.9367 - val_accuracy: 0.6412
Epoch 3/5
```

468/468 [=====] - 5420s 12s/step - loss: 0.9892 - accuracy: 0.4216 - val_loss: 0.9347 - val_accuracy: 0.6412

Model Using Embeddings from Corpus



GloVe Embeddings

Model architecture

Model: "sequential_1"

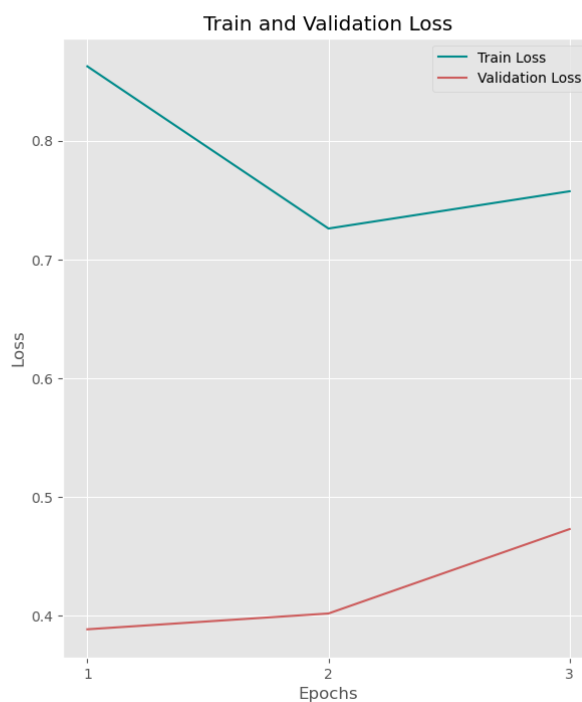
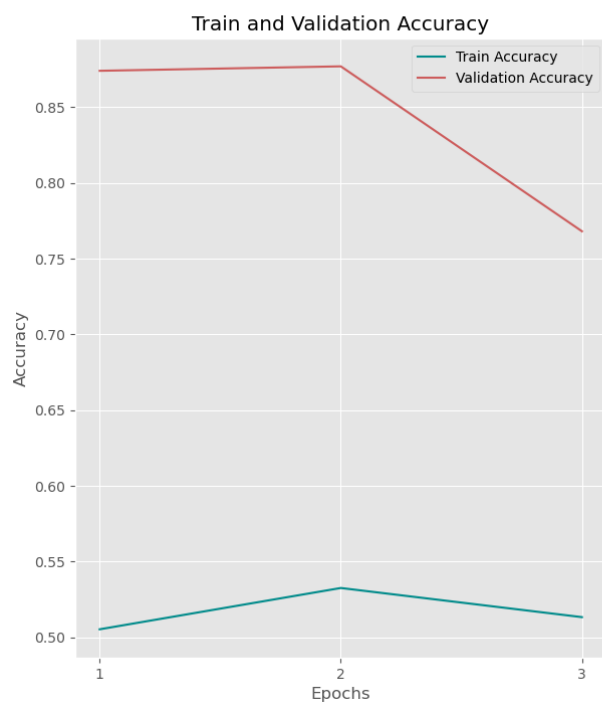
Layer (type)	Output Shape	Param #
=====		
embedding_1 (Embedding)	(None, 500, 300)	7054800
spatial_dropout1d_1 (Spatial Dropout)	(None, 500, 300)	0
bidirectional_1 (Bidirectional)	(None, 600)	1442400
dense_2 (Dense)	(None, 1024)	615424
dropout_2 (Dropout)	(None, 1024)	0
dense_3 (Dense)	(None, 3)	3075
dropout_3 (Dropout)	(None, 3)	0
activation_1 (Activation)	(None, 3)	0

```
=====
Total params: 9,115,699
Trainable params: 9,115,699
Non-trainable params: 0
```

Accuracy and loss scores

```
Epoch 1/5
468/468 [=====] - 5843s 12s/step - loss: 0.8627 - accuracy: 0.5053 - val_loss: 0.3883 - val_accuracy: 0.8739
Epoch 2/5
468/468 [=====] - 5379s 11s/step - loss: 0.7260 - accuracy: 0.5325 - val_loss: 0.4016 - val_accuracy: 0.8768
Epoch 3/5
468/468 [=====] - 8445s 18s/step - loss: 0.7574 - accuracy: 0.5133 - val_loss: 0.4727 - val_accuracy: 0.7680
```

Model Using Pre-Trained GloVe Word Vectors



Key Findings and Insights

The LSTM models were rather simple and originally set to train on 5 epochs, but the early stopping callback cut the training short as further epochs would not increase the performance of the models. Only 5 epochs were set due to limitations in computing resources. Deep learning models typically perform better when trained on larger datasets and perhaps this is why the GloVe embedding worked much better for the LSTM models compared to embeddings from the original corpus. Traditional machine learning models fared better and stochastic gradient descent trained on count vectorized data produced the best results. However, no determination was made as far as how these models would underfit or overfit on new data. The dataset itself is problematic because the text of the reviews is translated from a non-English language, so for even the best model, if a review from the United States was to be classified, it could be more likely to predict an inaccurate label. In such a case, a model trained on GloVe embeddings may perform better.

Next Steps

A clear obstacle was the nature of the text data, how they are translated to English from a non-English language and many uncommon grammatical errors are present. A similar untranslated dataset in English would be more suitable. The LSTM models could be improved and perhaps be made more sophisticated. A significantly larger dataset could be used. A model using a different recurrent neural network suited for NLP like GRU can be trained. Also, rather than use three labels of “positive,” “negative,” and “neutral,” the rating score from 1 to 5 can be used as labels. This would provide a slightly more informative idea of sentiment based on text from each review.